

Aufgabe 1 (Strategie – Vier Gewinnt).

Spielen Sie in Ihrer Gruppe das Spiel „Vier Gewinnt“ und versuchen Sie, Strategien bzw. Heuristiken für gute Spielzüge zu entwickeln und zu notieren. Unter <https://kimaster.mni.thm.de/> finden Sie eine Online-Plattform für das Spiel.

Notieren Sie im folgenden ihre ersten Strategien, die sie herausgefunden haben. Ihre Strategie sollte für bestimmte Zustände Punkte verteilen, z.B. falls direkt gewonnen werden kann ist der Spielzustand maximal gut.

- Direkter Gewinnzug: +1000 Punkte
- ...

Aufgabe 2 (Implementierung - Vier Gewinnt Heuristik).

In der Datei `ConnectFour.py` findet sich eine partielle Implementierung eines Vier Gewinnt Spiels. Es kann wie folgt instanziiert werden und gespielt werden. Aktuell ist eine sehr einfache AI implementiert: Der Computerspieler macht immer einen zufälligen, validen Zug.

```
In [ ]: %pip install ipywidgets IPython
```

```
In [1]: import numpy as np
import random
import sys

# Add the directory containing the module to Python path
module_dir = "/workspace/GdDS"
sys.path.append(module_dir)

import ConnectFour as cf
```

```
In [ ]: game = cf.ConnectFour()
cf.start_ui(game)
```

Im folgenden sollen Sie Ihre Heuristik implementieren und dagegen spielen. Die Heuristik soll alle möglichen nächsten Züge bewerten, und den besten Zug zurückgeben. Schauen Sie in die Datei `ConnectFour.py`. Implementieren Sie benötigte Helfermethoden. Eine erste Heuristik ist skizziert: Aktuell wird getestet ob ein Zug gewinnen kann, und dieser wird mit 1000 Punkten bewertet und direkt zurückgegeben.

```
In [ ]: class HeuristicConnectFour(cf.ConnectFour):
    def heuristic(self):
        best_score = -float("inf")
        best_move = None
        player = self.current_player
        for move in self.get_valid_moves():
            win = self.make_move(move)
            if win:
                self.undo_move(move)
```

```

        best_score = 1000
        return move, best_score

    score = 0

    # Ihre Bewertungen hier ...

    if score > best_score:
        best_score = score
        best_move = move

    return best_move, best_score

def get_best_move(self, depth=4):
    best_move, score = self.heuristic()
    return best_move

```

```

In [ ]: game = HeuristicConnectFour()
        cf.start_ui(game)

```

Aufgabe 3 (BetaMax).

Implementieren Sie nun Anhand ihrer Heuristik einen BetaMinMax Algorithmus. Hierfür müssen Sie die Methoden `beta_min` und `beta_max` implementieren. Benutzen Sie ihre implementierte Methode `heuristic` für die Bewertung von Spielzuständen.

```

In [ ]: class MinMaxConnectFour(HeuristicConnectFour):
        def _check_win_state(self):
            for row in range(self.n_row):
                for col in range(self.n_col):
                    if self.board[row, col] != 0 and self._check_win(row, col):
                        return True
            return False

        def beta_max(self, depth, alpha, beta):
            """
            BetaMax (Maximizer's turn) function of the Alpha-Beta pruning algorithm.

            Parameters:
            - depth: The maximum depth to search in the game tree.
            - alpha: The best score the maximizer can guarantee so far.
            - beta: The best score the minimizer can guarantee so far.

            Returns:
            - The best score for the maximizer.
            """
            pass

        def beta_min(self, depth, alpha, beta):
            """
            BetaMin (Minimizer's turn) function of the Alpha-Beta pruning algorithm.

            Parameters:
            - depth: The maximum depth to search in the game tree.
            - alpha: The best score the maximizer can guarantee so far.
            - beta: The best score the minimizer can guarantee so far.

            Returns:

```

```

- The best score for the minimizer.
"""

pass

def _get_best_move_min_max(self, depth=4):
    best_move = None
    best_value = -np.inf
    valid_moves = self.get_valid_moves()
    if len(valid_moves) == 0:
        return None

    for col in valid_moves:
        row = self._get_row_for_move(col)
        self.board[row, col] = self.PLAYER_1
        move_value = self.beta_min(depth - 1, -np.inf, np.inf)
        self.board[row, col] = 0

        if move_value > best_value:
            best_value = move_value
            best_move = col

    return best_move

def get_best_move(self, depth=4):
    move = self._get_best_move_min_max(depth=depth)
    if move is None:
        valid = self.get_valid_moves()
        move = valid[0] if len(valid) > 0 else 0
    return np.array(move)

```

```

In [ ]: game = MinMaxConnectFour()
        cf.start_ui(game)

```

Aufgabe 4: Ihre KI gegen KI-Master

Lassen Sie Ihre KI gegen den KI-Master antreten und werten Sie die Ergebnisse aus. Hierfür müssen Sie keinen weiteren Code implementieren. Testen Sie verschiedene Schwierigkeitsstufen. Passen Sie den Parameter `depth` an. Abhängig von der genauen Implementierung kann es sein, dass Teile in `KiMaster.py` angepasst werden müssen. Nach einer Anpassung in `KiMaster.py` müssen Sie den aktuellen Kernel neu starten (Kernel -> Restart Kernel).

```

In [ ]: import KiMaster as km

```

```

In [ ]: # Easy
        game = MinMaxConnectFour()
        await km.run(game, difficulty="easy", depth=4)

```

Bewertung:

```

In [ ]: # Medium
        game = MinMaxConnectFour()
        await km.run(game, difficulty="medium", depth=4)

```

Bewertung:

```
In [ ]: # Hard  
game = MinMaxConnectFour()  
await km.run(game, difficulty="hard", depth=4)
```

Bewertung: